

Ros2 Day4 hw2 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. 실행결과

1. 프로젝트 개요

이 프로젝트는 라이프사이클 노드와 QoS 설정을 적용한 퍼블리셔와 서브스크라이버다. 지금까지 만든 노드들은 실행하면 바로 동작을 시작했지만, 이번에는 상태를 나누어 두고 외부에서 상태를 바꿔줘야 동작하도록 만들었다.

라이프사이클 노드는 설정 전, 설정됨, 동작 중 같은 상태를 가진다. 실행만 하면 아무것도 하지 않고 기다리고 있다가, 설정 단계에서 발행자나 타이머를 만들고 활성화 단계로 넘어가야 실제로 발행을 시작한다. 반대로 비활성화하면 발행을 멈추고, 정리 단계에서는 만들어둔 것들을 해제한다.

토픽은 네 개를 쓴다. 문자열, 정수, 실수, 불리언을 각각 다른 토픽으로 발행하고 구독하며, 토픽마다 QoS를 다르게 설정했다. QoS는 데이터를 얼마나 확실하게 주고받을지, 나중에 들어온 노드에게 이전 값을 줄지 같은 통신 방식을 정하는 설정이다.

launch 파일은 두 개를 만들었다. 하나는 상태 전환을 직접 명령으로 해보기 위해 노드만 띄우는 것이고, 다른 하나는 상태 전환까지 자동으로 진행되도록 한 것이다.

2. 코드 구조

```
class MyPublisher : public rclcpp_lifecycle::LifecycleNode
{
public:
    using CallbackReturn=
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;
    ...
    CallbackReturn on_configure(const rclcpp_lifecycle::State &);
    CallbackReturn on_activate(const rclcpp_lifecycle::State &);
    CallbackReturn on_deactivate(const rclcpp_lifecycle::State &);
    CallbackReturn on_cleanup(const rclcpp_lifecycle::State &);
    CallbackReturn on_shutdown(const rclcpp_lifecycle::State &);
```

일반 노드 대신 라이프사이클 노드를 상속받고, 상태가 바뀔 때 실행될 함수들을 선언하는 부분. 각 함수는 성공 여부를 돌려주고, 실패를 반환하면 그 상태로 넘어가지 않는다.

```
rclcpp::QoS int_qos(rclcpp::KeepLast(10));
int_qos.reliable();
int_qos.transient_local();
```

```
rclcpp::QoS float_qos(rclcpp::KeepLast(5));
float_qos.best_effort();
float_qos.durability_volatile();
```

토픽마다 QoS를 다르게 설정하는 부분. 앞의 숫자는 몇 개까지 보관할지를 정하고, 확실하게 보낼지 빠르게 보낼지, 늦게 들어온 노드에게 이전 값을 줄지를 각각 지정한다. 정수 쪽은 확실하게 보내고 이전 값도 남기도록 했고, 실수 쪽은 값이 계속 바뀌는 것이라 빠른 쪽을 택했다.

```
int_pub_ = this->create_publisher<std_msgs::msg::Int32>("my_int_topic", int_qos);
...
timer_ = this->create_wall_timer(1s, std::bind(&MyPublisher::timer_callback, this));

RCLCPP_INFO(get_logger(), "on_configure");
return CallbackReturn::SUCCESS;
```

설정 단계에서 발행자 네 개와 타이머를 만드는 부분. 생성자가 아니라 여기서 만드는 것이 라이프사이클 노드의 핵심이다. 만들기만 하고 아직 발행은 하지 않는다.

```
MyPublisher::CallbackReturn MyPublisher::on_activate(const rclcpp_lifecycle::State & state)
{
    LifecycleNode::on_activate(state);
    RCLCPP_INFO(get_logger(), "on_activate");
    return CallbackReturn::SUCCESS;
}
```

활성화 단계. 첫 줄에서 부모 쪽 함수를 불러줘야 발행자들이 실제로 커진다. 이 줄이 없으면 상태는 바뀌지만 발행은 되지 않는다.

```
MyPublisher::CallbackReturn MyPublisher::on_cleanup(const rclcpp_lifecycle::State &)
{
    timer_.reset();
    str_pub_.reset();
    ...
}
```

정리 단계에서 만들어둔 것들을 해제하는 부분. 설정 단계에서 만든 것을 여기서 되돌린다고 보면 된다.

```
void MyPublisher::timer_callback()
{
    if (!str_pub_->is_activated())
    {
        RCLCPP_INFO(this->get_logger(), "inactive - not published");
        return;
    }
    ...
    msg.data = "Hello World: " + std::to_string(count_++);
    int_msg.data = count_;
    ...
}
```

1초마다 실행되는 부분. 타이머는 설정 단계에서 만들어져서 계속 돌지만, 활성화 상태가 아니면 발행하지 않고 로그만 남기고 빠져나온다. 덕분에 상태에 따라 동작이 달라지는 것을 로그로 확인할 수 있다.

```

MySubscriber::CallbackReturn MySubscriber::on_activate(const rclcpp_lifecycle::State &)
{
    ...
    my_str_subscriber_ = this->create_subscription<std_msgs::msg::String>(
        "my_str_topic",
        str_qos,
        std::bind(&MySubscriber::topic_callback, this, std::placeholders::_1));
    ...
}

```

구독자 쪽은 활성화 단계에서 구독을 만든다. 발행자와 달리 구독자는 켜고 끄는 기능이 따로 없어서, 활성화될 때 만들고 비활성화될 때 해제하는 방식으로 처리했다.

```

MySubscriber::CallbackReturn MySubscriber::on_deactivate(const rclcpp_lifecycle::State &)
{
    my_str_subscriber_.reset();
    ...
}

```

비활성화 단계에서 구독을 해제하는 부분. 이렇게 해야 비활성화 상태에서 메시지를 받지 않는다.

```

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    rclcpp::executors::SingleThreadedExecutor exe;
    auto node = std::make_shared<MyPublisher>();
    exe.add_node(node->get_node_base_interface());
    exe.spin();
    ...
}

```

메인 함수. 라이프사이클 노드는 지금까지처럼 노드를 그대로 넘길 수 없어서, 실행기를 따로 만들고 노드의 기본 인터페이스를 넘겨준다.

```

pub = LifecycleNode(
    package='tongsin2',
    executable='my_publisher',
    name='my_publisher',

```

```
namespace=",
output='screen')
```

수동 전환용 launch 파일. 노드를 띄우기만 하고 상태는 바꾸지 않는다. 실행한 뒤 터미널에서 명령으로 하나씩 전환해보기 위한 것이다.

```
launch.actions.RegisterEventHandler(
    launch.event_handlers.on_process_start.OnProcessStart(
        target_action=talker_node,
        on_start=[
            launch.actions.EmitEvent(event=launch_ros.events.lifecycle.ChangeState(
                lifecycle_node_matcher=launch.events.matches_action(talker_node),
                transition_id=lifecycle_msgs.msg.Transition.TRANSITION_CONFIGURE,
            )),
        ],
    ),
),
```

자동 전환용 launch 파일의 한 부분. 노드가 실행되면 설정 전환을 보내라는 뜻이다. 아래로는 설정이 끝나면 활성화, 활성화되고 5초 뒤에 비활성화, 그 다음 정리와 종료까지 이어지도록 같은 형태를 나열했다. 구독자 쪽은 설정과 활성화까지만 하도록 했다.

3. 파일 구조

```
tongsin2
├── include
│   ├── my_publisher.hpp
│   └── my_subscriber.hpp
├── src
│   ├── my_publisher.cpp
│   └── my_subscriber.cpp
├── launch
│   ├── manual_transition.launch.py
│   └── auto_transition.launch.py
├── CMakeLists.txt
└── package.xml
```

(4. 실행결과 첨부 영상 참고)